

Kidney-RAG — Day 2 Teammate Guide

Retrieval Optimization · Creativa × Orange AI Hackathon

How to set up the project, what to build for Day 2, and how to use AI to do it well.

This guide is for the two teammates joining the retrieval-optimization work. The ingestion pipeline (parsing → chunking → embedding → hybrid retrieval) is already built and on GitHub. Day 2 is about making it **provably good**: measured Precision@k, a justified top-k, a benchmarked embedding model, and a chunk-size experiment — each backed by numbers, not opinion.

What already exists (do NOT rebuild)

Component	File	State
PDF parser (4 guidelines)	pdf_parser.ipynb	Done
Section-aware chunker (666 chunks)	chunker.ipynb	Done
Embedder + ChromaDB index	embedder.ipynb	Done
Hybrid retriever (semantic + BM25)	retriever.ipynb	Done
Labeled eval set (18 questions)	eval/eval_set.json	Done

The corpus is 4 official public guidelines: KDIGO 2024 CKD, NICE NG203, KDIGO 2022 Diabetes-in-CKD, and USPSTF CKD Screening. 666 chunks, embedded with the medical model abhinand/MedEmbed-large-v0.1 (1024-dim), stored in ChromaDB.

Part 1 — Get the project running (15 min)

Step 1: Clone the repo

```
git clone https://github.com/mohamed-mahmoud-de/Kidney-RAG.git
cd Kidney-RAG
```

Step 2: Create a virtual environment + install dependencies

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
# Mac/Linux:
source .venv/bin/activate

pip install -r requirements.txt
```

Step 3: Rebuild the vector index locally

The Chroma index and embeddings are **not** in git (they are large and regenerable). Rebuild them from the committed chunks. The first run downloads the MedEmbed model (~1.3 GB, one time) and embeds 666 chunks (~15–20 min on CPU):

```
python -m jupyter nbconvert --to notebook --execute embedder.ipynb --inplace
```

This writes corpus/chunks/chunk_embeddings.npy and corpus/chunks/chroma_db/. After the first run the model is cached at ~/.cache/huggingface/ — **never delete that folder** or it re-downloads 1.3 GB.

Step 4: Confirm retrieval works

```
python -m jupyter nbconvert --to notebook --execute retriever.ipynb --inplace
```

Open the executed retriever.ipynb — you should see clinical queries returning ranked chunks with document, section, page, and similarity score.

Golden rule for notebooks

Always run with --inplace. Never use --output other.ipynb — it creates duplicate files that confuse the repo. Read CLAUDE.md at the repo root: it documents every pipeline decision, the chunk schema, and the ‘things to never

do' list.

Part 2 — Folder structure (know where things live)

```
Kidney-RAG/
|-- CLAUDE.md                <- READ FIRST: all pipeline decisions + rules
|-- requirements.txt         <- pip install -r this
|-- pdf_parser.ipynb        <- PDFs -> corpus/parsed/*.json
|-- chunker.ipynb           <- parsed -> corpus/chunks/all_chunks.jsonl
|-- embedder.ipynb          <- chunks -> embeddings + chroma_db (index)
|-- retriever.ipynb         <- hybrid search (semantic + BM25)
|-- eval/
|   \-- eval_set.json        <- 18 labeled questions + gold chunk_ids
|-- corpus/
|   |-- raw_pdfs/            <- the 4 source guideline PDFs
|   |-- parsed/*.json       <- clean page-level text (parser output)
|   |-- chunks/
|       |-- all_chunks.jsonl <- 666 chunks (THE shared source of truth)
|       |-- chunk_embeddings.npy (gitignored, regenerable)
|       |-- chroma_db/       (gitignored, regenerable)
|   |-- SOURCES.md          <- source list + licenses
|   |-- manifest.json
\-- docs/                  <- decks + this guide
```

Where your new work goes: put new experiment notebooks at the repo root (e.g. `evaluator.ipynb`, `ablation.ipynb`) and write any result logs/CSVs into a new `eval/results/` folder. Never hand-edit `all_chunks.jsonl` — it is generated by the chunker.

Part 3 — Your Day-2 tasks (split between the two of you)

The Day-2 deck requires: a justified top-k, a chunk-size experiment, Precision@k for ≥ 2 embedding models, logged comparisons, and retrieved chunks visible before generation. Split it into two tracks so you work in parallel without colliding.

TRACK A — Evaluation & Metrics (owner: teammate 1)

Goal: turn 'looks good' into real numbers using `eval/eval_set.json`.

- **Build** `evaluator.ipynb`. Load `eval_set.json` + the retriever functions. For each question, retrieve top-k and compare returned `chunk_ids` against `gold_chunk_ids`.
- **Compute 3 metrics** averaged over the set: Precision@k = (gold chunks in top-k)/k; Recall@k = (gold in top-k)/(total gold); Hit@k = 1 if any gold in top-k.
- **Compare cosine-only vs hybrid** (both already exist in `retriever.ipynb`). Which wins on Precision@5?
- **Justify top-k**: compute the metrics at k = 3, 5, 10 and pick the best trade-off. The deck suggests starting at k=4.
- **Handle out-of-scope (q16-q18) separately**: gold is empty; the 'right' answer is a LOW top similarity. Report the max similarity for these — it sets the refusal threshold for Day 4.
- **Log everything** to `eval/results/metrics.csv` and print a summary table.

TRACK B — Model & Chunk Experiments (owner: teammate 2)

Goal: prove the model and chunk-size choices with evidence (a deck requirement).

- **Embedding head-to-head (required)**: build `model_compare.ipynb`. Re-embed the same 666 chunks with a SECOND model — use BAAI/bge-large-en-v1.5 (general) vs our MedEmbed-large (medical). Score both on `eval_set.json` with Precision@5. Expected story: the medical model wins — prove it.
- **Chunk-size ablation (required)**: build `ablation.ipynb`. Re-chunk the corpus at 3 configs (e.g. 400/50, 550/55, 700/100 size/overlap), re-index each, score on the eval set. To keep it fast, use a small model like all-MiniLM-L6-v2 for the ABLATION only (relative comparison), then keep MedEmbed for the final index.
- **Log results** in the deck's table format: config | Precision@5 | notes. Write to `eval/results/`.
- **Do NOT overwrite the main index** while experimenting — point experiments at a temp Chroma path (e.g. `corpus/chunks/chroma_tmp/`).

Shared (either of you)

- **Explainability view:** a simple Streamlit app or even a clean notebook table that, for a typed query, shows each retrieved chunk's text + similarity score + (document, section, page). The deck says Retrieval Quality (30 pts) is easiest to demo when it is visible.

Part 4 — How to use AI effectively

AI (Claude / ChatGPT / Cursor / Claude Code) will speed you up a lot — if you give it the right context. The project already contains everything it needs to understand the codebase.

The 3-step workflow that works

- **1. Feed it context first.** Paste the contents of CLAUDE.md and the relevant notebook (e.g. retriever.ipynb) into the chat. Say: 'This is our medical RAG project. Here is how it works.' Never ask it to write code blind.
- **2. Give it the exact task + the data shape.** e.g. 'Write evaluator.ipynb. Input: eval/eval_set.json (schema: question, gold_chunk_ids). Use the hybrid_search(query,k) function from retriever.ipynb. Compute Precision@k, Recall@k, Hit@k averaged over the set.'
- **3. Verify every number by hand.** AI will confidently produce wrong metrics. Pick ONE question, trace it manually (what did retrieval return? is the gold chunk in there?), and confirm the script agrees. This is a medical system — never trust an unchecked number.

Good prompt templates (copy these)

Track A:

```
"Here is our chunk schema and the retriever code [paste].
Write evaluator.ipynb that loads eval/eval_set.json, runs
hybrid_search and cosine_search for k in [3,5,10], and
reports Precision@k, Recall@k, Hit@k per category and
overall. Save a CSV to eval/results/metrics.csv."
```

Track B:

```
"Using sentence-transformers, write model_compare.ipynb that
embeds corpus/chunks/all_chunks.jsonl with BOTH
abhinand/MedEmbed-large-v0.1 and BAAI/bge-large-en-v1.5,
builds a separate Chroma collection for each, and scores
both on eval/eval_set.json with Precision@5."
```

AI rules for THIS project (safety)

- **Never let AI invent clinical facts or citations.** Every answer must trace to a real chunk_id from retrieval. If it writes a threshold or dose from memory, delete it.
- **Never change the chunk schema** (chunk_id, document_name, source_url, page_number, page_range, section_title, token_count, text). Everything downstream depends on it.
- **Keep MedEmbed as the query + document model together.** If you swap the embedding model, you must re-embed both chunks and queries with the same model.
- **Ask AI to explain, not just generate.** 'Why did you pick this k?' catches bad assumptions before they reach the demo.

Part 5 — Git workflow

- Pull latest before you start: `git pull origin main`
- Work on a branch: `git checkout -b track-a-eval` (or track-b-experiments)
- Commit your notebook + result files, NOT the chroma_db / .npy (they are gitignored).
- Push and open a pull request: `git push -u origin your-branch`

```
git pull origin main
git checkout -b track-a-eval
# ... build evaluator.ipynb, run it ...
git add evaluator.ipynb eval/results/
git commit -m "Add evaluator: Precision@k for cosine vs hybrid"
git push -u origin track-a-eval
```

Definition of Done (Day-2 checklist)

#	Deliverable	Owner
1	Top-k chosen and justified with Precision@k	Track A

2	cosine vs hybrid compared on the eval set	Track A
3	Out-of-scope max-similarity reported (refusal threshold)	Track A
4	Precision@5 for ≥ 2 embedding models, documented choice	Track B
5	Chunk-size/overlap ablation run and logged	Track B
6	Retrieved chunks visible before generation (basic UI)	Shared

Pitfalls the judges have seen before — avoid these: tuning k or chunk size by gut instead of measured Precision@ k ; testing only easy queries (our eval set deliberately includes the hard albuminuria-threshold case and 3 out-of-scope refusals); comparing on one metric only (note latency too); and hiding retrieval from the demo. Every claim on Day 5 should trace to a number in `eval/results/`.

Repo: <https://github.com/mohamed-mahmoud-de/Kidney-RAG> · Questions → check CLAUDE.md first.